Practical ways to reduce token usage = Save Money

1. Pre-process before calling GPT

Created by Negash

Only call GPT if your program can't answer with rules or logic.

```
def classify_text(text):
    """Classify the input text into a category.
    rule-based shortcuts for common cases"""

    if "refund" in text.lower():
        return "billing_issue"

     """ Only call GPT for more complex cases."""
    return call_gpt(text)


def call_gpt(prompt):
    return client.responses.create(
        model="gpt-5.3",
        input=prompt
    )
```

2. Use conditional prompts

Send prompts only when they are meaningful or necessary.

```
def generate_summary(text):
    """Generate a summary of the given text using GPT.
    If the text is too short, return it as is."""
    if len(text) < 50:  # too short to summarize
        return text
    return call_gpt(f"Summarize:\n{text}")
```

3. Cache GPT responses

Reuse answers for repeated inputs.

```
cache = {}
def cached_gpt(prompt):
    """Call the GPT model with caching to avoid redundant calls."""
    if prompt in cache:
        return cache[prompt]
        # Created by Negash
    response = call_gpt(prompt)  # Replace with actual GPT call
    cache[prompt] = response
    return response
```

4. Save and reuse common outputs

Pre-store common explanations instead of regenerating them.

```
COMMON_RESPONSES = {
    "password_reset": "To reset your password, click 'Forgot Password'...",
    "refund_policy": "Refunds are processed within 5-7 business days..."
}

def get_response(intent):
    '''Return a response based on the user's intent.'''
    if intent in COMMON_RESPONSES:
        return COMMON_RESPONSES[intent]

    return call_gpt(f"Explain {intent}")
```

5. Batch GPT calls.

Send multiple inputs in one request instead of many small ones.

```
def batch_summarize(texts):
    """Summarize a batch of texts using GPT."""
    prompt = "Summarize each:\n" + "\n---\n".join(texts)

    return call_gpt(prompt)

texts = ["Text 1...", "Text 2...", "Text 3..."]
batch_summarize(texts)
```

6. Limit GPT usage in the UI layer

Avoid calling GPT on every user interaction (e.g., typing).

```
"""Handle user submission and call the GPT model."""
def on_submit(user_input):
    """Only trigger after user submits, not on every keystroke"""
    return call_gpt(user_input)
```

7. Trim prompt size (very important)

```python
def smart_prompt(context):
    """Generate a prompt for GPT based on the conversation context.
        Keep only last 2 messages"""
    trimmed = context[-2:]
    """Join the last 2 messages into a single string and call GPT with it."""
    return call_gpt("\n".join(trimmed))
```

8. Use shorter instructions.

Avoid verbose system prompts.

Every prompt (system + user + assistant history) is converted into tokens
Longer system prompt = more tokens consumed per request.

If you send the same system prompt with every API call, you pay for it every time.
If you're making many repeated calls, shortening the system prompt can save a lot.

```
# Bad (long)
prompt = "You are a highly intelligent assistant that provides detailed..."

# Better (short)
prompt = "Summarize clearly:"
```

9. Limit max output tokens

Control response length explicitly.

The model will only use as many tokens as it needs (up to the limit)

Use a lower limit to reduce cost, but base it on risk to avoid cutting off responses.

```
""" set 100 tokens for the response."""
response = client.responses.create(
    model="gpt-5.3",
    input="Explain recursion",
    max_output_tokens=100
)
```

10. Use cheaper models for simple tasks

```python
def route_task(task):
    """Route a task to the appropriate model based on the task type."""
    if task == "simple":
        model = "gpt-5-mini"
    else:
        model = "gpt-5.3"
   # This is a placeholder for the actual implementation of the client and responses.
    return client.responses.create(model=model, input="Do task")
```